

Apache Spark

Advanced Topics — Pure Theory Reference

Volume Contents

Chapter 1: Spark Performance Tuning

- 1.1 The Performance Bottleneck Hierarchy
- 1.2 Partitioning and Parallelism
- 1.3 Data Skew — Causes, Detection, and Remedies
- 1.4 Memory Architecture and the Unified Memory Manager
- 1.5 Caching and Persistence Strategy
- 1.6 Serialisation
- 1.7 Shuffle — The Most Expensive Operation
- 1.8 I/O Optimisation — File Formats and Partitioned Storage
- 1.9 The Catalyst Optimizer and Code Generation
- 1.10 Adaptive Query Execution (AQE)
- 1.11 Summary of Tuning Parameters

Chapter 2: Stream Processing and Structured Streaming

- 2.1 Stream Processing Fundamentals
- 2.2 Batch vs Stream Processing — A Deep Comparison
- 2.3 Key Challenges in Streaming Systems
- 2.4 Delivery Guarantees — At-Most-Once, At-Least-Once, Exactly-Once
- 2.5 Event-Time vs Processing Time vs Ingestion Time
- 2.6 Stateful Processing — Concepts and Types
- 2.7 Windows on Event Time
- 2.8 Tumbling Windows
- 2.9 Sliding Windows
- 2.10 Session Windows
- 2.11 Handling Late Data with Watermarks
- 2.12 Output Modes — Append, Update, Complete
- 2.13 Dropping Duplicates in a Stream
- 2.14 Structured Streaming — Core Architecture
- 2.15 The Unbounded Table Model
- 2.16 Micro-Batch vs Continuous Processing
- 2.17 Fault Tolerance and Checkpointing
- 2.18 Trigger Types
- 2.19 Transformations on Streams
- 2.20 Input Sources and Output Sinks

Chapter 1: Spark Performance Tuning

Performance tuning is one of the most critical and nuanced skills for any Spark practitioner. Apache Spark runs in a complex, distributed environment where many variables interact: network latency, memory pressure, CPU utilisation, disk I/O, JVM garbage collection, data distribution, and query plan quality. A job that runs in 5 minutes on a well-tuned cluster might take 5 hours on a poorly configured one with the same hardware.

The foundation of all good performance tuning is a simple principle: measure first, then fix. The Spark UI provides rich metrics — jobs, stages, tasks, shuffle bytes, GC time, and memory usage — that tell you exactly where time is being spent. Guessing and blindly applying configurations is unreliable and can actually make performance worse. Always profile the job, identify the actual bottleneck, fix that specific bottleneck, and then measure again to confirm improvement.

1.1 The Performance Bottleneck Hierarchy

In practice, Spark performance problems almost always stem from one of five root causes. They are listed below in order of typical impact — data skew is by far the most common and most impactful issue in production Spark environments:

1. Data Skew

Data skew occurs when the data is not evenly distributed across partitions. Some partitions end up much larger than others because certain keys in the dataset are far more frequent. For example, in an e-commerce dataset, a few large retail customers might have millions of orders while thousands of small customers have just a handful. When Spark groups or joins data by `customer_id`, the reducer handling the large customers processes millions of records while all other reducers finish in seconds. The entire stage cannot complete until the slowest task finishes, so one slow task poisons the performance of the whole job.

Data skew is typically visible in the Spark UI's Stages tab: the task duration distribution shows one or a few tasks with dramatically longer execution times than all others. The shuffle read bytes column will also show one task reading far more data than the rest.

2. Shuffle Overhead

A shuffle is the physical redistribution of data across partitions and executor nodes. It is required by wide transformations such as `groupBy`, `join`, `distinct`, and `repartition`. Every shuffle involves serialising data to disk on the source executors, transferring it across the network to the destination executors, and deserialising it again. This combination of disk I/O, network I/O, and serialisation/deserialisation makes shuffles the most expensive operations in Spark.

Jobs with many shuffle stages, or shuffles involving very large datasets, are often dominated by shuffle time. The Spark UI shows shuffle read and write bytes per stage, making it straightforward to identify heavy shuffles.

3. Memory Pressure

When executor memory is insufficient for the computation being performed, Spark must spill data to disk. Spilling is far slower than in-memory computation and can turn a fast in-memory job into a slow

disk-bound one. In severe cases, when even the disk spill cannot accommodate the data, Spark tasks fail with Out of Memory (OOM) errors. Memory pressure also causes the JVM's garbage collector to run frequently, pausing all computation on that executor while it reclaims memory. High GC time is visible in the Executors tab of the Spark UI.

4. Insufficient Parallelism / I/O Inefficiency

If a Spark job reads far more data than it needs (because filters are not pushed down to the storage layer, or because row-oriented formats are used instead of columnar formats), the I/O cost dominates. Similarly, if the number of partitions is too small, the job underutilises the available CPU cores — some cores sit idle while others are busy. Conversely, too many tiny partitions creates excessive task scheduling overhead.

5. CPU / Serialisation Inefficiency

In PySpark, every Python User-Defined Function (UDF) requires data to be serialised from the JVM, transferred to the Python interpreter, processed, and serialised back. This serialisation round-trip can be 10 to 100 times slower than equivalent native Spark SQL operations that stay within the JVM. Poor choice of serialisation library (Java's default serialiser vs Kryo) also adds unnecessary overhead.

1.2 Partitioning and Parallelism

Definition:

A partition is a logical chunk of a dataset stored on one executor node. Each task processes exactly one partition. The number of partitions therefore directly determines the degree of parallelism — how many tasks can run concurrently.

Why Partitioning Matters

Consider a cluster with 20 executor cores. If a DataFrame has only 5 partitions, only 5 of the 20 cores can be active at any time — 75% of the cluster is idle. Increasing to 80 partitions means all 20 cores can be busy with 4 tasks each, fully utilising the hardware. On the other hand, if there are 10,000 tiny partitions, the task scheduling overhead (each task involves driver-executor communication, JVM startup for the task, and result reporting) can dominate over the actual computation.

Rules for Choosing Partition Count

The general guideline is 2 to 4 partitions per available CPU core. If a cluster has 10 executors with 4 cores each (40 total cores), the target partition count for post-shuffle stages is 80 to 160. For the default shuffle partition count (configured via `spark.sql.shuffle.partitions`, default 200), this is often the first parameter tuned.

A second guideline focuses on partition size: each task should process approximately 100 to 200 megabytes of data. If a dataset is 20 gigabytes, the optimal number of partitions is roughly 100 to 200. If partitions are much smaller (say, 1 MB each), the task overhead is wasteful. If they are much larger (several gigabytes), tasks take too long and the risk of OOM increases.

Types of Partitioning

- **Hash Partitioning:** The default for most wide transformations. Each record is assigned to a partition by computing a hash of the key column(s) modulo the number of partitions. Records with the same key always go to the same partition — which is necessary for correct groupBy and join results. Hash partitioning is prone to data skew when certain keys are very frequent.
- **Range Partitioning:** Partitions data into ordered, contiguous ranges of key values. Useful for sort operations and for writing ordered data to storage. Less prone to skew than hash partitioning for uniformly-distributed keys.
- **Round-Robin Partitioning:** Distributes records uniformly across partitions without regard to their key values. Used by repartition() without a column argument. Produces balanced partitions but destroys any grouping by key.
- **Custom Partitioning:** In RDD-based programming, users can implement a custom Partitioner class to define exactly which partition each record belongs to. Useful for domain-specific optimisation.

Key Concept:

Pre-partitioning two DataFrames by the same join key before caching them can eliminate shuffle entirely in subsequent joins between those DataFrames. This is one of the highest-impact optimisations for repeated joins in iterative algorithms.

1.3 Data Skew — Causes, Detection, and Remedies

Root Causes of Data Skew

Data skew arises from the natural non-uniformity of real-world data. In a customer database, a handful of enterprise customers may generate orders of magnitude more transactions than the majority of customers. In a log dataset, null values are often grouped into a single partition because null hashes to the same value in all hash partitioning schemes. In a geospatial dataset, events in a major city far outnumber events in rural areas.

Detecting Data Skew

The most reliable way to detect skew is through the Spark UI. In the Stages tab, examine the Task Duration column and the Shuffle Read Size column for any stage that involves a wide transformation. Signs of skew include:

- The maximum task duration is many times larger (10x, 100x) than the median task duration.
- One or a few tasks show vastly larger shuffle read sizes than all others.
- The stage takes much longer than expected given the total data size and cluster capacity.
- CPU utilisation drops dramatically towards the end of a stage as all but one or two tasks complete.

Remedies for Data Skew

The two most effective remedies are Adaptive Query Execution (AQE) and Salting:

Adaptive Query Execution (AQE)

In Spark 3.0 and later, AQE can detect and automatically handle skewed partitions without any code changes. When AQE's skew join optimisation is enabled, Spark detects partitions that are significantly larger than the median (by a configurable factor) and splits them into multiple smaller sub-partitions. The corresponding partition from the other side of the join is replicated to match each sub-partition. This effectively distributes the work of the large partition across multiple tasks. AQE is the recommended first remedy to try.

Salting

Salting is a manual technique for distributing skewed data. The idea is to artificially diversify a skewed key by appending a random integer (the 'salt') to it. For example, if the key 'NULL' causes all null-keyed records to go to one partition, you can append a random number between 0 and 9 to create 10 different salted keys (NULL_0, NULL_1, ..., NULL_9), distributing the records across 10 partitions. On the other side of the join, the non-skewed table must be replicated — one copy for each salt value — so that matching can still occur. Salting eliminates skew but increases the size of the smaller join side by the salt factor.

Handling Null Keys Separately

A common practical approach is to split the dataset into two DataFrames: one containing the skewed keys (often nulls or known heavy-hitter keys) and one containing the rest. The two subsets are processed separately and the results are combined with a union. This avoids the overhead of salting on the non-skewed portion.

1.4 Memory Architecture and the Unified Memory Manager

Understanding how Spark allocates and manages memory within an executor JVM is essential for diagnosing and fixing memory-related problems.

The Executor Memory Regions

Memory Region	Size (Default)	Purpose	Tuning Parameter
Reserved Memory	300 MB fixed	Internal Spark system structures. Not user-accessible.	Not tunable
User Memory	40% of (heap – 300 MB)	User-defined data structures, metadata, Python objects in PySpark, UDF state.	spark.memory.fraction (inverse)
Unified Memory Pool	60% of (heap – 300 MB)	Shared pool for both Execution Memory and Storage Memory. Dynamic boundary between the two.	spark.memory.fraction = 0.6
Execution Memory (within Unified)	Dynamically claimed	Buffers for shuffles, joins, sorts, and aggregations in progress.	spark.memory.storageFraction (inverse)
Storage Memory (within Unified)	Dynamically claimed	Cached / persisted RDDs and DataFrames.	spark.memory.storageFraction = 0.5

Dynamic Memory Sharing

Before Spark 1.6, execution and storage memory were statically separated. This was wasteful: if a job used no caching, the storage memory sat idle, and conversely, if a job did heavy caching, execution memory could be starved. The Unified Memory Manager, introduced in Spark 1.6, replaced this with a single shared pool where execution and storage memory coexist with a dynamic boundary.

When execution memory needs more space for a shuffle and storage memory has unused capacity, execution can borrow from storage. However, execution can also forcibly evict cached data from storage if it needs space. Storage can borrow from execution only when execution memory is idle. This dynamic allocation means the available memory adapts to the actual workload pattern.

Off-Heap Memory

JVM on-heap memory is subject to garbage collection, which periodically pauses all threads on an executor while the GC reclaims unreachable objects. For large, long-lived objects (such as shuffled data or cached DataFrames), this GC pressure can significantly degrade performance. Spark's Tungsten engine can store data in off-heap memory — directly managed binary memory outside the JVM's heap — which bypasses GC entirely. Off-heap memory is configured separately and must be explicitly enabled. It is used automatically by DataFrames for binary columnar storage.

1.5 Caching and Persistence Strategy

Definition:

Caching (or persisting) a dataset means instructing Spark to store the computed result of a transformation in memory or on disk so that subsequent operations on the same dataset can reuse it without recomputing from scratch.

When to Cache

Caching is beneficial only when the cost of storing the data is outweighed by the savings from avoiding recomputation. The clearest case is when a dataset is used multiple times in the same application. If a filtered, aggregated DataFrame is used as input to three different downstream computations, Spark would otherwise recompute it from scratch three times — reading the original files, applying all the transformations, and repeating the same work. Caching avoids this duplication.

Caching is also valuable for datasets that are expensive to compute (many joins, aggregations, wide transformations) but much smaller than the raw input — for instance, a heavily filtered and aggregated summary table derived from a multi-terabyte log file.

When Not to Cache

Caching a dataset that is only used once wastes executor memory without any benefit. The overhead of writing to the cache (serialisation, memory allocation) is a cost that must be paid, and if there is no reuse, the benefit never materialises. Similarly, caching very large datasets that exceed available executor memory causes thrashing — cached data is partially evicted to disk, and the cache hit rate drops, potentially making performance worse than not caching at all.

Storage Levels

Storage Level	Uses Memory?	Uses Disk?	Replicated?	Best Scenario
MEMORY_ONLY	Yes (JVM objects)	No	No	Dataset fits comfortably; recomputation is cheap
MEMORY_AND_DISK	Yes	Spill if needed	No	Default for DataFrames; safe when size is uncertain
DISK_ONLY	No	Yes	No	Very large datasets; access latency is acceptable
MEMORY_ONLY_SER	Yes (serialised bytes)	No	No	Saves memory at cost of CPU for serialisation
MEMORY_ONLY_2	Yes	No	Yes (x2)	Critical data; can tolerate executor loss without recomputation
OFF_HEAP	Yes (off JVM)	No	No	Avoid GC overhead; requires Tungsten off-heap enabled

Insight:

For DataFrames, MEMORY_AND_DISK is the safe default because Spark's Tungsten binary format is much more compact than JVM object graphs. A DataFrame that appears to be 10 GB in JVM objects may only use 2 GB in Tungsten binary format, making it much more likely to fit in memory.

1.6 Serialisation

Serialisation is the process of converting an in-memory object into a stream of bytes so that it can be transmitted over a network or written to disk, and deserialisation is the reverse process. In a distributed system like Spark, serialisation happens constantly: when task code is sent from the driver to executors, when data is shuffled between executors, when broadcast variables are distributed, and when data is spilled to disk.

Java Serialisation (Default)

Java's built-in serialisation mechanism is used by default in Spark for RDD-based workloads. It is highly compatible — it can serialise virtually any Java or Scala object without configuration — but it is also slow and produces large byte streams. The byte stream includes extensive type metadata and object graph information that is redundant for Spark's purposes. Java serialisation is suitable for development and debugging but should not be used in production for performance-sensitive workloads.

Kryo Serialisation

Kryo is a third-party Java serialisation library that is significantly faster than Java serialisation (typically 3 to 10 times faster) and produces much smaller byte streams (typically 3 to 5 times smaller). Kryo achieves this efficiency by using a compact binary format and avoiding redundant type metadata for registered classes. Spark recommends Kryo for production use and makes it straightforward to enable. When classes are explicitly registered with Kryo, serialisation and deserialisation are even faster because Kryo can use numeric class IDs instead of full class names. For unregistered classes, Kryo falls back to a slightly less efficient but still fast mode.

Tungsten Binary Format (Automatic for DataFrames)

DataFrames and Datasets use Spark's internal Tungsten binary format, which is separate from both Java and Kryo serialisation. Tungsten stores data in a compact, CPU-cache-friendly binary layout directly in memory (either on-heap or off-heap). The format is designed to maximise CPU efficiency by enabling vectorised operations, avoiding unnecessary object creation, and enabling data to be processed directly in its binary form without deserialisation. DataFrames automatically benefit from Tungsten without any configuration.

Remember:

The performance hierarchy for data representation in Spark: Tungsten binary (DataFrames) > Kryo serialised (RDDs with Kryo) > Java serialised (RDDs with default). Always prefer DataFrames over RDDs for structured data — you get Tungsten for free.

1.7 Shuffle — The Most Expensive Operation

A shuffle is the redistribution of data across partitions and network nodes that is required by wide transformations. It is the single most expensive operation in Spark because it involves multiple costly steps: serialisation of data to disk on the source executor (the shuffle write), network transfer of that data to the destination executor, and deserialisation and sorting on the destination executor (the shuffle read).

The Anatomy of a Shuffle

During a shuffle, the processing pipeline is divided into two stages by a stage boundary. In the first stage (the map stage), each task computes its output and writes it to local disk in a set of shuffle files, partitioned by the target partition they belong to. In the second stage (the reduce stage), each task fetches the shuffle data it is responsible for from the disk of all other executors over the network, combines it (if applicable), and processes it. The map stage and reduce stage cannot overlap — all map tasks must complete before any reduce tasks begin reading their shuffle input.

Strategies to Minimise Shuffle Cost

There are several ways to reduce the cost of shuffles:

Map-Side Combine (combineByKey / reduceByKey)

When performing a keyed aggregation (such as computing a sum per key), it is possible to partially aggregate data within each partition before shuffling. Instead of shuffling all individual values and aggregating at the reducer, the executor first combines all values for a given key within its local partition, then sends only the partial aggregate to the reducer. This dramatically reduces the volume of data shuffled. The groupByKey operation in RDD programming does NOT perform map-side combine — it shuffles all individual values, which is why reduceByKey is always preferred for aggregations.

Broadcast Join — Eliminating Shuffle Entirely

When one side of a join is small enough to fit in memory on each executor, it can be broadcast — a copy is sent to every executor and stored in memory. The larger side of the join is then processed locally on each executor by looking up keys in the in-memory copy of the smaller side. Because all data stays local, no shuffle is needed at all. This is the single biggest win available for join performance when one side is small. Spark automatically applies broadcast joins when one side is smaller than the autoBroadcastJoinThreshold configuration (default 10 MB, often raised to 50–200 MB in practice).

Pre-partitioning and Bucketing

If two large tables that are frequently joined together are stored pre-partitioned by the join key (either by repartitioning and caching them, or by writing them as bucketed Hive tables), Spark can skip the shuffle entirely on subsequent joins because matching partitions from both tables already reside on the same executor. Bucketing (writing data to storage with a fixed, predetermined number of hash partitions by a chosen column) is the persistent version of this pattern — the benefit persists across Spark sessions without needing to repartition at runtime.

1.8 I/O Optimisation — File Formats and Partitioned Storage

Columnar vs Row-Oriented Storage

The choice of file format has a profound impact on query performance for analytical workloads. Row-oriented formats (CSV, JSON) store data one record at a time: all fields of record 1 come first, then all fields of record 2, and so on. When a query accesses only 3 columns out of 100, a row-oriented format still reads all 100 columns for every row, wasting 97% of the I/O.

Columnar formats (Parquet, ORC) store data one column at a time: all values of column 1 are stored together, then all values of column 2, etc. A query accessing 3 columns out of 100 reads only those 3 columns' data — a potential 97% reduction in I/O. Columnar formats also achieve much better compression because values within a single column tend to be of the same type and have similar values, making them highly compressible.

Format	Orientation	Column Pruning	Predicate Pushdown	Schema Embedded?	Recommendation
CSV	Row	No	No	No	Avoid for analytics
JSON	Row	Partial	No	No	Avoid for large data
Parquet	Columnar	Yes	Yes (row groups)	Yes	Primary choice
ORC	Columnar	Yes	Yes (stripes)	Yes	Good for Hive

Delta Lake	Columnar+ACID	Yes	Yes	Yes	Best for mutable data
Avro	Row (binary)	No	No	Yes	Good for Kafka messages

Predicate Pushdown

Predicate pushdown is an optimisation where a filter condition (predicate) is 'pushed down' to the data source level, so that only records matching the filter are read from storage into Spark. For Parquet files, statistics (minimum and maximum values, null counts) are stored in each row group's metadata. When a filter is applied (e.g., `score > 80`), Spark reads these statistics for each row group and skips entire row groups that cannot contain matching records, without reading a single byte of their actual data. This can eliminate large fractions of I/O for highly selective filters.

Partition Pruning (Physical Partitioning)

When a dataset is written to storage partitioned by one or more columns (e.g., by department and year), the data is organised into a directory hierarchy on disk. A query that filters on the partition columns can skip entire directories — Spark simply does not open the files in non-matching directories. For example, if data is partitioned by year and the query filters `WHERE year = 2024`, only the `year=2024/` directory is read. All other year directories are completely ignored. This is called partition pruning and can eliminate enormous amounts of I/O for date-partitioned event logs.

1.9 The Catalyst Optimizer and Code Generation

What is the Catalyst Optimizer?

The Catalyst Optimizer is the query optimisation engine built into Spark SQL and the DataFrame API. When a user writes a DataFrame query or SQL statement, Catalyst transforms it through a series of phases to produce the most efficient physical execution plan possible. Catalyst operates on a tree representation of the query: the query is parsed into a tree of logical nodes, each representing an operation (scan, filter, project, join, aggregate), and Catalyst applies transformation rules to this tree to produce a more efficient tree.

The Four Phases of Catalyst

1. **Analysis Phase:** The unresolved logical plan (column names and table references as strings) is resolved by looking up column names, function signatures, and table schemas in the Catalog. The output is a fully resolved logical plan with all types determined.
2. **Logical Optimisation Phase:** A set of rule-based transformations is applied to the resolved plan. Key optimisations include: predicate pushdown (moving filters closer to data sources), column pruning (removing unused columns from scan operations), constant folding (evaluating constant expressions at compile time), and expression simplification (e.g., simplifying `'NOT (a AND b)'` to `'(NOT a) OR (NOT b)'` via De Morgan's law).
3. **Physical Planning Phase:** The optimised logical plan is converted into one or more physical plans, each specifying exactly how the computation will be executed (which join algorithm, how many

shuffle partitions, etc.). A cost-based optimiser estimates the cost of each physical plan using table statistics (row counts, column cardinalities, data sizes) and selects the lowest-cost plan.

4. **Code Generation Phase:** The Tungsten engine generates optimised JVM bytecode for the selected physical plan. The key technique is Whole-Stage Code Generation, which fuses multiple operators (filter + project + hash aggregate) into a single generated function, eliminating the overhead of virtual function calls and iterator-based processing between operators. The generated code can approach the performance of hand-written Java code.

Key Concept:

The Catalyst Optimizer is the primary reason why DataFrame operations are faster than equivalent RDD operations for structured data. With RDDs, Spark executes exactly what you write. With DataFrames, Catalyst can completely reorganise your query into a more efficient form before executing it.

1.10 Adaptive Query Execution (AQE)

Traditional query optimisation (including Catalyst's Logical Optimisation phase) happens before any data is processed, based on static statistics that may be inaccurate or absent. Adaptive Query Execution, introduced in Spark 3.0, extends optimisation to runtime: Spark collects actual statistics from completed shuffle map stages and uses them to re-optimize the remaining plan. This closes the gap between what the optimiser expected and what the data actually looks like.

Three Key AQE Capabilities

1. Dynamic Partition Coalescing

After a shuffle completes, Spark knows the actual sizes of all shuffle output partitions. AQE can merge small adjacent partitions into larger ones, reducing the number of post-shuffle tasks. This is particularly valuable because the optimal number of shuffle partitions depends on the data size, which varies between stages and between runs. Instead of guessing a fixed `spark.sql.shuffle.partitions` value, AQE dynamically adjusts the partition count based on actual data. This prevents the 'too many small tasks' problem that commonly occurs in multi-stage jobs where intermediate results are much smaller than the original data.

2. Dynamic Join Strategy Switching

When planning a join, Spark may estimate that both sides are large and choose a Sort Merge Join (which requires a shuffle). But at runtime, after the map stage of the join completes, AQE may discover that one side is actually small enough to broadcast. AQE can switch the join strategy from Sort Merge Join to Broadcast Hash Join, eliminating the shuffle entirely — even though the query plan was already executing.

3. Dynamic Skew Join Handling

AQE can detect skewed partitions in a join by comparing each partition's size to the median partition size. When a partition is more than a configurable factor larger than the median (default: 5 times), AQE splits it into multiple smaller sub-partitions and replicates the corresponding partition from the other side of the join. The skewed work is then distributed across multiple tasks instead of one, eliminating the bottleneck. This is automatic salting without any code changes.

1.11 Summary of Key Tuning Parameters

Parameter	Default	What It Controls	Typical Tuning Advice
spark.sql.shuffle.partitions	200	Number of partitions for post-shuffle stages in DataFrames / SQL	Set to 2–4× total executor cores; AQE can auto-tune
spark.sql.adaptive.enabled	true (Spark 3.2+)	Enables Adaptive Query Execution	Always leave enabled; disable only for debugging
spark.sql.autoBroadcastJoinThreshold	10 MB	Maximum size for automatic broadcast join	Raise to 50–200 MB if memory allows; set to -1 to disable
spark.executor.memory	1g	JVM heap memory per executor	Set based on available RAM minus OS overhead; 4–16 GB typical
spark.memory.fraction	0.6	Fraction of executor heap for Unified Memory Pool	Increase to 0.75–0.8 for memory-intensive jobs
spark.serializer	Java	Serialisation library for shuffle and broadcast	Always set to KryoSerializer for production RDD workloads
spark.executor.cores	1 (YARN)	CPU cores per executor	Set to 4–5 for balance of parallelism and memory per task
spark.default.parallelism	Cluster-dependent	Default number of partitions for RDD operations	Set to 2–3× total executor cores

Chapter 2: Stream Processing and Structured Streaming

Stream processing is the continuous computation of results over data that arrives unboundedly over time. Rather than accumulating a finite dataset and running a batch job over it, a streaming system processes each record — or small batches of records — as they arrive, producing continuously updated results.

Apache Spark's Structured Streaming engine, introduced in Spark 2.0, brings the power of the Spark SQL engine and its optimisations to stream processing. It provides a unified, high-level API for both batch and streaming workloads, strong end-to-end fault tolerance guarantees, event-time processing with late-data handling, and flexible stateful computation.

2.1 Stream Processing Fundamentals

Definition:

Stream processing is the paradigm of processing data continuously and incrementally as it arrives from one or more sources, rather than processing a complete, finite dataset all at once (as in batch processing).

The concept of a data stream is abstractly simple: a sequence of records, each carrying a payload (the event data) and a timestamp, that arrives continuously and without a predetermined end. In practice, streams originate from diverse sources: web server access logs, IoT sensor readings, financial market tick data, social media posts, payment transactions, user clickstream events, and system telemetry.

The processing system consuming a stream must operate continuously, maintaining its own internal state across records, managing memory carefully (since data volume is unbounded), and tolerating failures without losing data or producing incorrect results. These constraints make stream processing fundamentally more complex than batch processing.

2.2 Batch vs Stream Processing — A Deep Comparison

Dimension	Batch Processing	Stream Processing
Data Arrival	All data is available upfront. The dataset is complete and finite before processing begins.	Data arrives continuously without a predetermined end. The dataset is unbounded and always growing.
Processing Model	A job is submitted, runs to completion over the full dataset, and terminates.	A long-running query continuously processes new data as it arrives, never (intentionally) terminating.
Latency	Minutes to hours. Acceptable for end-of-day reports, data warehouse loads, ML training.	Milliseconds to seconds. Required for real-time dashboards, fraud detection, alerting systems.
Throughput	Very high. Can optimise globally for bulk processing over all data at once.	Moderate. Must balance throughput with latency requirements.
State Management	State is materialised at the end of the job. The entire dataset is available for reference.	State must be maintained incrementally across events, bounded in size, and fault-tolerant.

Fault Recovery	Restart the job from scratch (or from a checkpoint) on failure.	Resume from the last committed offset/checkpoint without data loss or duplication.
Result Freshness	Data is stale by the time the batch job completes (hours old).	Results are continuously updated with minimal delay from event occurrence.
Operational Complexity	Relatively simple. Finite data, clear start and end.	High. Unbounded data, out-of-order events, late data, state growth, exactly-once semantics.
Typical Use Cases	ETL pipelines, data warehouse loads, ML model training, historical analysis, compliance reporting.	Fraud detection, live dashboards, real-time recommendations, alerting, session analytics, IoT monitoring.

2.3 Key Challenges in Streaming Systems

2.3.1 Unbounded Data

The most fundamental challenge of stream processing is that the input dataset has no end. This makes many otherwise-simple operations impossible or meaningless. Sorting an infinite stream has no defined output (you cannot produce a sorted result until you have seen all the data). Computing an exact distinct count requires remembering every element ever seen. Computing a global average requires knowing the total sum and total count for all records ever — which consumes unbounded memory.

Stream processing systems address this challenge by restricting computation to bounded time windows (e.g., 'count per 5-minute window'), by using approximate algorithms (e.g., HyperLogLog for approximate distinct counts), or by defining 'since beginning' aggregations that continuously update a running result.

2.3.2 Out-of-Order and Late Data

In a distributed system, events rarely arrive at the processing engine in the order they were generated. A user action that occurred at 14:03:00 on a mobile device may arrive at the Kafka broker at 14:03:45 because of a slow network connection. Another user action from 14:02:55 may arrive at 14:05:00 because the device was temporarily offline and the event was buffered. When Spark processes a micro-batch at 14:05:30, it receives the 14:02:55 event alongside current events — but it has already processed the 14:00–14:05 window at 14:05:00.

The question the system must answer is: what do we do with this late event? Ignoring it produces incorrect results (undercount). Waiting indefinitely for all late events means results are never produced. A practical compromise is the watermark: a configurable threshold that defines the maximum expected delay. Events that arrive later than this threshold are considered too late and are discarded; events that arrive within the threshold are accepted even if they are late.

2.3.3 State Management

Many useful streaming computations are inherently stateful. Counting events per user requires remembering the current count for every user. Computing average transaction amount per merchant requires maintaining a running sum and count per merchant. Detecting fraud patterns requires remembering the sequence of recent transactions per account. Session analytics requires tracking when each user's session started and how many events it has contained.

This state must be stored somewhere, must survive executor failures (fault tolerance), must not grow without bound (bounded state), and must be accessible quickly during processing (low-latency access). Spark Structured Streaming uses a distributed state store — backed by either an in-memory hash map or RocksDB (an embedded key-value store) — that is periodically checkpointed to durable storage (HDFS or S3).

2.3.4 Exactly-Once Semantics

In a distributed system, failures are not exceptional — they are routine. Executors crash, network partitions occur, and applications are restarted for upgrades. When a failure occurs in the middle of processing a micro-batch, the system must decide what to do: re-process the batch (risking duplicates) or skip it (risking data loss). Neither outcome is acceptable for correctness-critical applications like financial transaction processing.

Achieving exactly-once semantics — where each event affects the output exactly once, regardless of failures — requires coordination between the streaming engine, the source, and the sink. The source must provide a way to replay data from a specific point (Kafka offsets, file names). The engine must maintain a durable, atomic record of which data has been processed. The sink must either support idempotent writes (writing the same data twice has the same effect as writing it once) or transactional writes (writing is atomic and either succeeds completely or has no effect).

2.4 Delivery Guarantees

Guarantee	Mechanism	Risk	Appropriate When
At-Most-Once	Events are processed at most once. If a failure occurs, events may be lost but never reprocessed.	Data loss — some events may never be processed.	Metrics sampling, debugging logs, cases where occasional data loss is acceptable.
At-Least-Once	Events are never lost — on failure, they are re-processed from the last checkpoint. Duplicates are possible.	Duplicate processing — some events may be processed more than once.	When data loss is unacceptable and duplicates can be handled by idempotent operations or downstream deduplication.
Exactly-Once	Each event is processed exactly once. No loss, no duplication. Requires idempotent or transactional sinks.	Complexity — harder to implement; requires careful source, engine, and sink coordination.	Financial transactions, billing, inventory management, any correctness-critical pipeline.

Key Concept:

Spark Structured Streaming achieves end-to-end exactly-once semantics through the combination of: (a) replayable sources (Kafka, file systems) that can re-deliver data from a specific offset on restart, (b) a write-ahead offset log that durably records which data has been read, and (c) idempotent or transactional output sinks.

2.5 Event-Time vs Processing Time vs Ingestion Time

This distinction is one of the most important conceptual foundations of stream processing, and a source of frequent confusion and bugs in streaming applications.

Time Concept	Definition	Assigned By	Key Challenge
Event Time	The timestamp when the event actually occurred in the real world — when the user clicked, when the sensor measured, when the transaction was executed.	The device, application, or sensor that generated the event. Embedded in the event payload.	Events arrive late and out of order. Requires watermarks to handle correctly.
Ingestion Time	The timestamp when the event arrived at the messaging system (e.g., the Kafka broker received it).	The messaging broker (e.g., Kafka's log append timestamp).	Reflects network delays, not actual event timing. More accurate than processing time for ordering.
Processing Time	The timestamp when Spark's streaming engine actually processed the event in a micro-batch.	The Spark engine itself (essentially the system clock at the time of processing).	Reflects system latency and lag. Completely divorced from real-world event timing. Wrong for business logic.

Why Event Time is Almost Always Correct

Consider a streaming analytics system counting the number of purchases per hour for a retail dashboard. The purpose is to answer the question: 'How many purchases happened between 14:00 and 15:00?' Processing time is useless for this question because it tells you when Spark ran, not when purchases occurred. If Spark's job is running 10 minutes behind real time (a common occurrence during traffic spikes), an event that occurred at 14:59 might be processed at 15:09 and incorrectly attributed to the 15:00–16:00 window. Event time, embedded in the event payload, always correctly reflects when the purchase happened.

The trade-off is complexity: because event time can be late (an event from 14:59 might arrive at 15:30), the system must decide how long to wait before closing the 14:00–15:00 window. This is the function of watermarks.

2.6 Stateful Processing — Concepts and Types

Definition:

Stateful processing is any stream computation where the output for an event depends not only on the event itself but also on previously-seen events. The system must maintain and update some form of accumulated state across the processing of individual events.

The State Store

Spark Structured Streaming manages state in a distributed state store. By default, the state store is an in-memory hash map (one per executor, partitioned by key). For larger state or more durable state management, RocksDB can be used as the state backend — RocksDB is an embedded key-value store

that can handle terabytes of state on local SSDs. The state store is periodically snapshotted (checkpointed) to durable storage (HDFS or S3), enabling recovery after executor or application failures. Every stateful operation in Structured Streaming is implicitly associated with the state store. When a micro-batch processes new events for a key, it reads the current state for that key from the state store, updates it based on the new events, and writes the updated state back. The checkpoint mechanism ensures this state persists across failures.

Types of Stateful Operations

- **Windowed Aggregation:** The most common stateful operation. For each key and each time window, Spark maintains an accumulator (count, sum, max, etc.) in the state store. As new events arrive, the corresponding window accumulators are updated. When the watermark advances past a window's end time, the window's result is finalised, emitted to the sink, and its state is evicted from the store.
- **Stream Deduplication:** Spark maintains a set (or hash table) of event identifiers seen within the watermark window. Incoming events are checked against this set. If the ID is already present, the event is a duplicate and is discarded. If it is new, it is processed and its ID is added to the set. Watermarks bound the size of this set by evicting old IDs.
- **Stream-Stream Join:** When joining two streaming DataFrames, both streams' events must be buffered in the state store until matching events from the other stream arrive. Watermarks on both streams define how long events are buffered before being discarded as too old to match.
- **Custom Stateful Operations (`flatMapGroupsWithState` / `applyInPandasWithState`):** Arbitrary, user-defined state per key. The user provides a function that receives the current state and a batch of new events for a key, updates the state, and emits output rows. Supports state timeouts. Used for session tracking, complex event processing, and machine learning inference over streams.

Important:

Custom state operations (`flatMapGroupsWithState`) can grow the state store indefinitely if state is never evicted. Always define a state timeout — either `ProcessingTimeTimeout` (the state expires a fixed duration after the last update) or `EventTimeTimeout` (the state expires when the watermark advances past a defined event time threshold). Unbounded state store growth will eventually exhaust executor memory and crash the streaming job.

INSTITUTE OF TECHNOLOGY

2.7 Windows on Event Time — Overview

A window is a bounded, time-delimited grouping of events. Window aggregations allow streaming systems to compute aggregates over a finite, contiguous period of time — answering questions like 'how many transactions occurred in the last 5 minutes' or 'what was the average sensor reading over the past hour, updated every 15 minutes?'

In Spark Structured Streaming, windows are always defined on the event time column, not on processing time. This ensures that the window boundaries correspond to real-world time, regardless of when events arrive at the processing engine. The `window()` function takes the event time column and either one or two duration arguments, depending on the window type.

Key Concept:

All window aggregations in Structured Streaming require a watermark to be defined on the event time column. Without a watermark, Spark cannot know when a window is complete (since late events might still arrive), and so it must retain state for all windows indefinitely — causing unbounded state growth. The watermark tells Spark when it is safe to finalise a window and evict its state.

2.8 Tumbling Windows

Definition:

A tumbling window is a fixed-duration, non-overlapping, contiguous window. Windows tile the time axis without gaps and without overlap, like non-overlapping tiles on a floor. Every event belongs to exactly one tumbling window.

Characteristics

- **Fixed Duration:** Every tumbling window has the same length (e.g., 5 minutes, 1 hour, 1 day).
- **Non-Overlapping:** Adjacent windows share no events. A window ends precisely where the next one begins.
- **One Event, One Window:** Each event is assigned to exactly one window based on its event time.
- **Deterministic:** Given an event's timestamp, its window membership is always the same regardless of processing order or arrival time.
- **Epoch-Aligned by Default:** Windows are aligned to the Unix epoch (midnight UTC on 1 January 1970) unless a `startTime` offset is specified. A 1-hour window will always start at full hours (00:00, 01:00, 02:00, etc.).

How a Tumbling Window is Computed

Conceptually, when Spark processes events with a tumbling window aggregation, it assigns each event to a window by computing which window interval the event's timestamp falls into. For a 5-minute tumbling window, an event with timestamp 14:07:32 falls into the window [14:05:00, 14:10:00). Spark maintains an accumulator per (key, window) pair in the state store — for example, ('merchant_A', [14:05, 14:10)) → {count: 47, sum: 12450.00}.

As new events arrive (including late events within the watermark threshold), the corresponding accumulators are updated. When the watermark advances past 14:10:00 (signalling that no more events for the [14:05, 14:10) window can arrive), the window is finalised: its accumulated result is emitted to the output sink, and its state is evicted from the store.

Use Cases

- Hourly or daily summary reports (total sales per hour, daily active users).
- Per-interval error rate monitoring (errors per 5-minute window).
- IoT data aggregation (average temperature per 10-minute interval per sensor).
- Billing and metering (total API calls per hour per customer).

2.9 Sliding Windows

Definition:

A sliding window is a window of fixed duration that advances by a configurable step interval smaller than the window size. Because the step is smaller than the window size, consecutive windows overlap, and an event can belong to multiple windows simultaneously.

Characteristics

- **Fixed Duration:** Every sliding window has the same length, just like a tumbling window.
- **Configurable Slide:** The window slides forward by a fixed step at each trigger. When $\text{step} < \text{duration}$, windows overlap.
- **One Event, Multiple Windows:** An event with timestamp T belongs to all windows that contain T . For a 1-hour window sliding every 15 minutes, an event belongs to 4 windows (the four consecutive 1-hour windows that contain its timestamp).
- **Higher State Cost:** Because each event contributes to multiple windows, the state store must maintain more accumulators than for tumbling windows. The ratio of window size to slide step determines the multiplication factor.

Example

A 1-hour sliding window advancing every 15 minutes computes a rolling 1-hour aggregate that is updated four times per hour. An event at 14:30 belongs to the windows [13:30–14:30], [13:45–14:45], [14:00–15:00], and [14:15–15:15]. Each of these four accumulators is updated when the event is processed. This is useful for computing metrics like 'requests per hour, updated every 15 minutes' — a rolling window that gives more granular visibility than a tumbling hourly window.

Use Cases

- Rolling averages and moving statistics (e.g., 30-day rolling average revenue updated daily).
- Trend detection over sliding periods (e.g., is the error rate in the last hour trending upward?).
- Sliding window anomaly detection (e.g., rate-of-change alerts).

Insight:

Sliding windows increase memory consumption proportionally to $(\text{window_size} / \text{slide_step})$. A 1-hour window sliding every 1 minute means each event contributes to 60 accumulators. For large numbers of keys and frequent slides, this can stress the state store significantly. Use sliding windows sparingly and always pair with appropriately sized watermarks.

2.10 Session Windows

Definition:

A session window groups events based on periods of activity separated by gaps of inactivity. A session is a sequence of events where the gap between consecutive events does not exceed a configured maximum gap duration. A new session starts when the gap since the last event exceeds this threshold.

Characteristics

- **Variable Duration:** Unlike tumbling and sliding windows which have fixed, predetermined durations, session windows can be arbitrarily short (a burst of activity) or long (an extended interaction). Their length is determined by the data.
- **Data-Driven Boundaries:** A session ends not at a fixed clock time but when a configurable period of inactivity has elapsed. If a user generates events at 14:00, 14:02, 14:07, then nothing until 14:52, two sessions might be created: [14:00–14:07] and [14:52–...], if the gap threshold is 30 minutes.
- **Suitable for User Interaction Modelling:** Session windows are the natural model for web analytics (user browsing sessions), IoT device activity (a device is 'on' when events arrive, 'off' during gaps), and any domain where bursty activity separated by idle periods is meaningful.
- **More Complex State Management:** Because session boundaries are not predetermined, the state for a session must be maintained dynamically. Two events that initially appear to belong to different sessions may merge into one session if a bridging event arrives late.

Use Cases

- Web analytics — tracking user sessions (sequence of page views bounded by 30-minute inactivity).
- IoT device activity — determining how long a machine was actively running vs idle.
- Customer journey analysis — grouping a sequence of purchase-related events into a single shopping trip.
- Fraud detection — identifying suspicious sequences of transactions within a short time window.

2.11 Handling Late Data with Watermarks

Definition:

A watermark is a monotonically increasing threshold, expressed as a lag behind the maximum observed event time, below which events are considered too late to be processed and are discarded. Formally:
$$\text{watermark}(t) = \text{max_event_time_observed}(t) - \text{threshold_delay}.$$

The Purpose of Watermarks

Watermarks serve two critical purposes in Structured Streaming. First, they define when a window can be considered complete and its accumulated result can be safely emitted to the sink. Without a watermark, Spark cannot know whether more events for a window might still arrive — it must wait indefinitely, retaining state for all windows forever. With a watermark of 10 minutes, Spark can safely conclude that when the watermark advances past a window's end time, no more events for that window will arrive, and the window can be finalised.

Second, watermarks define the state retention policy. Once the watermark has advanced past a certain point in time, state for windows or keys that ended before that point can be safely evicted from the state store. Without this eviction mechanism, the state store would grow indefinitely as new windows and keys are encountered, eventually exhausting executor memory.

How the Watermark Advances

The watermark is maintained as a single value that tracks the maximum event time seen across all events in all partitions, minus the configured threshold. As each micro-batch is processed, Spark scans the event timestamps in the incoming data and updates the global maximum. The watermark is then set to this maximum minus the threshold. The watermark is monotonically increasing — it never decreases, even if newer micro-batches happen to contain only old events.

Example: Suppose the watermark threshold is 10 minutes. After processing the first batch (events with timestamps up to 14:08), the watermark is 14:08 – 10 min = 13:58. After the second batch (events up to 14:15), the watermark advances to 14:05. Windows ending before 14:05 can now be finalised and emitted.

The Threshold Trade-off

Setting a larger watermark threshold means: accepting more late events (fewer events are discarded), but retaining state for longer (higher memory consumption) and increasing the latency before windows are emitted (since the watermark advances more slowly relative to the current processing time). Setting a smaller threshold means: lower latency and smaller state, but more events are discarded as 'too late' and results may be less accurate.

The optimal threshold depends on the characteristics of the data source. For low-latency, well-connected sources, a 1–5 minute threshold may be sufficient. For mobile app events that might be buffered for hours when the device is offline, a threshold of several hours may be appropriate.

2.12 Output Modes — Append, Update, Complete

The output mode controls what data is written to the sink in each micro-batch. Different output modes are supported depending on whether the streaming query includes stateful aggregations and watermarks.

Output Mode	What is Written	When Applicable	Requires Watermark?
append	Only new rows that are final and will never change are written. For stateful aggregations, a row is only appended after the watermark passes its window end (guaranteeing it is complete). For stateless operations, every new record is appended immediately.	Stateless queries; windowed aggregations with watermark; deduplication with watermark.	Yes (for stateful ops)
update	Only rows that have changed since the last micro-batch are written. For stateful aggregations, intermediate (incomplete) window results are written as they are updated by new events. Suitable for sinks that support upserts (update-or-insert).	Most query types; particularly useful for low-latency database writes.	Not required
complete	The entire result table is re-	Global aggregations that	No (incompatible)

	written in every micro-batch. Because the entire output is rewritten each time, the sink must be able to replace all previous output. Only supported for aggregate queries without watermark.	must always reflect the complete result; small output tables.	with watermark agg)
--	---	---	---------------------

Remember:

The combination of append output mode with a watermark is the canonical pattern for windowed streaming analytics in production. Append mode guarantees that each window result is emitted only once, after it is complete, and will never be updated. This simplifies downstream consumers — they can treat the stream as a sequence of final results.

2.13 Dropping Duplicates in a Stream

Duplicate events are an unavoidable reality in distributed streaming systems. At-least-once delivery semantics in Apache Kafka (the most common streaming source) mean that a producer retry or a consumer restart can cause the same event to be delivered and processed more than once. Application restarts after a failure, combined with the offset log rollback required for recovery, can also cause re-processing of events near the restart boundary.

The Deduplication Challenge

Deduplication in a streaming context is more complex than in batch processing. In batch processing, you simply load the full dataset and apply a `DISTINCT` operation. In streaming, you must remember which event IDs you have already processed without retaining an ever-growing set of all IDs ever seen. The watermark mechanism addresses this: if you set a watermark of 1 hour on the event time column and use the `event_id` as the deduplication key, Spark maintains a set of event IDs seen within the past 1 hour. Events older than 1 hour are assumed to have already been processed and deduplicated; their IDs are evicted from the state store.

How Streaming Deduplication Works

Spark's `dropDuplicates()` operation in streaming mode maintains a state store keyed by the deduplication columns (e.g., `event_id`). For each incoming event in a micro-batch, Spark checks whether the `event_id` already exists in the state store. If it does, the event is a duplicate and is filtered out. If it does not, the event is new: it is passed to the next transformation and its `event_id` is added to the state store. The watermark drives state eviction: once the watermark advances past a certain point, event IDs with timestamps before that point are removed from the state store, preventing unbounded growth.

Key Considerations

- The watermark threshold must be at least as large as the maximum expected re-delivery delay. If Kafka can buffer unacknowledged messages for up to 2 hours before retrying, a 2-hour watermark is required to guarantee deduplication.

- The deduplication key should be a unique, stable identifier for each event (e.g., a UUID generated at the source, or a composite of source system ID + sequence number). Using non-unique identifiers will incorrectly discard valid distinct events.
- Without a watermark, the state store for deduplication grows indefinitely — every unique event ID ever seen is retained. This is only safe for very short-lived streams or streams with very low distinct event counts.

2.14 Structured Streaming — Core Architecture

Structured Streaming is Apache Spark's production-ready streaming engine, built on top of the Spark SQL engine. Its fundamental design decision is to expose a streaming computation as a series of small, incremental batch computations on new data. This approach, called micro-batch processing, allows Structured Streaming to reuse the entire Spark SQL optimisation stack (Catalyst, Tungsten, code generation) for streaming workloads.

Design Principles

- Unified API: The same DataFrame/Dataset and SQL API is used for both batch and streaming processing. A developer who knows Spark SQL can write streaming queries without learning a completely different API.
- Strong Fault Tolerance: End-to-end exactly-once semantics via write-ahead logging, offset management, and idempotent output. The system can recover from any failure without data loss or duplication.
- Incremental Computation: Each micro-batch processes only the new data that arrived since the previous micro-batch, not the entire historical dataset. Stateful operations maintain running state in the state store.
- Event-Time Support: Native support for event-time semantics, watermarks, and late-data handling, built directly into the engine.
- Source and Sink Abstraction: A clean abstraction separates the streaming engine from the specifics of reading from Kafka vs files and writing to Parquet vs databases. New connectors can be added without changing the engine.

2.15 The Unbounded Table Model

The central mental model of Structured Streaming is to treat a live data stream as an unbounded, continuously appending table. This is called the Input Table. Every new event that arrives from the source (Kafka, files, etc.) is conceptually appended as a new row to this table.

The streaming query is defined as a declarative query over this continuously growing input table, exactly as you would write a SQL query over a static table. The query engine continuously evaluates this query and maintains a Result Table that reflects the current state of the query's output. The output sink receives the changes to the Result Table according to the configured output mode (append, update, or complete).

This abstraction is powerful because it hides all the complexity of incremental computation, offset management, and state management from the query author. The developer simply describes what result they want; Structured Streaming figures out how to compute it incrementally and efficiently.

2.16 Micro-Batch vs Continuous Processing

Aspect	Micro-Batch Processing	Continuous Processing
How It Works	Spark periodically triggers a batch job that processes all data accumulated since the last trigger. The batch runs to completion, commits results, and updates checkpoints before the next batch begins.	Each record is processed as soon as it arrives, using a long-running pipeline. Checkpointing happens in epochs (time-based intervals) rather than per-batch.
Latency	Latency is bounded below by the trigger interval. Typical latency is 100 milliseconds to several seconds. Sub-100ms latency is not achievable.	Millisecond or sub-millisecond latency possible. Each record is processed almost immediately after arrival.
Throughput	Very high. Each batch is optimised as a complete unit, benefiting from all Spark SQL optimisations.	Moderate. The continuous pipeline is simpler but cannot apply batch-level optimisations like whole-stage codegen across operators.
Fault Tolerance	Exactly-once via per-batch checkpointing. Recovery is clean — restart from the last committed batch.	At-least-once by default. Exactly-once requires additional coordination. Recovery may cause brief duplication.
Supported Operations	All Structured Streaming operations are supported.	Limited — only stateless transformations (map, filter) and simple aggregations. No windowing, no stream-stream joins.
Production Status	Production-ready since Spark 2.0. The recommended mode for all production workloads.	Experimental as of Spark 3.x. Not recommended for production.

Insight:

For the vast majority of production use cases, micro-batch processing is the correct choice. It provides very high throughput, strong fault tolerance guarantees, and supports all streaming operations. Continuous processing should only be considered when end-to-end latency in the single-digit millisecond range is genuinely required and the workload is simple stateless transformations.

INSTITUTE OF TECHNOLOGY

2.17 Fault Tolerance and Checkpointing

Structured Streaming's fault tolerance mechanism is based on the concept of durable write-ahead logging combined with replayable sources and idempotent sinks. Together, these three components guarantee that the streaming computation can be recovered to an exact consistent state after any failure.

The Checkpoint Structure

The checkpoint directory (which must be located on durable, distributed storage — HDFS, Amazon S3, Azure Data Lake Storage) contains three components:

The Offset Log

The offset log records the exact range of source data (e.g., Kafka topic-partition-offset ranges, or the set of file paths) that was read in each micro-batch. It is written atomically before the batch begins processing. On restart after a failure, Spark reads the offset log to determine exactly where it left off and resumes reading from the source at that precise point — neither re-reading already-processed data nor skipping unprocessed data.

The Commit Log

The commit log records which micro-batches have been fully written to the sink and committed. On restart, Spark cross-references the offset log and the commit log to determine whether the most recent batch in the offset log was already committed. If it was committed, Spark starts from the next batch. If it was not committed (meaning the application failed after reading the batch but before fully writing the results), Spark re-processes the batch. Combined with an idempotent or transactional sink, this ensures exactly-once output.

State Store Checkpoint

For stateful operations (windowed aggregations, deduplication, stream-stream joins), the state store is periodically snapshotted to the checkpoint directory. Each snapshot captures the complete state at a specific micro-batch boundary. On restart, the latest state store snapshot is loaded, and the streaming query resumes processing from the corresponding offset in the offset log. This ensures that accumulated state (window accumulators, seen event IDs, buffered join events) is not lost on failure.

Requirements for Exactly-Once

- The source must be replayable — it must be able to re-deliver data from a specific offset (Kafka, file systems, Delta Lake all satisfy this).
- The checkpoint location must be on durable storage — not local disk, which is lost when an executor or node restarts.
- Each streaming query must have a unique checkpoint directory — sharing checkpoints between queries causes corruption.
- The sink must be idempotent (writing the same data twice has the same effect as writing once) or transactional (writes are atomic).

2.18 Trigger Types

Definition:

A trigger defines how frequently Spark initiates a new micro-batch to process available data. The trigger determines the latency/throughput trade-off of the streaming query.

- **Default Trigger (As Fast as Possible):** Spark starts a new micro-batch immediately after the previous one completes. There is no configured wait. This maximises throughput but does not guarantee any specific trigger interval. If each batch takes 500ms to process, the effective trigger rate is approximately 2 batches per second.
- **Fixed Interval Trigger:** A micro-batch is triggered at a fixed interval (e.g., every 5 seconds, every 1 minute). If data arrives faster than the trigger rate, it accumulates and is processed in larger

batches. If no new data arrives in an interval, no batch is triggered. This provides predictable, configurable latency at the cost of potentially processing data in larger, less frequent chunks.

- **Once Trigger** (Deprecated in Spark 3.3): Processes all available data in exactly one micro-batch and then stops the streaming query. This allows running a streaming query as if it were a batch job — useful for periodic incremental loads from Kafka or new files in a directory.
- **AvailableNow Trigger** (Spark 3.3+, Replaces Once): Similar to Once, but processes all available data in multiple optimal-sized micro-batches instead of one potentially very large batch. This is more memory-efficient and allows AQE to optimise each batch. The query stops automatically after all available data is processed.
- **Continuous Trigger** (Experimental): Enables continuous processing mode, where records are processed as they arrive without micro-batch overhead. The trigger interval here refers to the checkpoint interval, not the processing interval.

2.19 Transformations on Streams

Stateless Transformations

Stateless transformations process each event independently, without any reference to previously seen events. They do not require any state to be maintained between micro-batches. These transformations are fully supported in streaming mode without any additional constraints, and they behave identically whether applied to a batch DataFrame or a streaming DataFrame.

Examples of stateless transformations include: selecting specific columns (projection), filtering records based on a condition (selection), adding new computed columns (with functions like type casting, string manipulation, date extraction, mathematical operations), parsing embedded structured data (from JSON strings, for example), and applying stateless user-defined functions.

Stateful Transformations

Stateful transformations require information about previously seen events to produce correct output. They all require a watermark to bound the state that must be retained. The primary stateful transformations in Structured Streaming are windowed aggregations (`groupBy` with `window()`), deduplication (`dropDuplicates()`), stream-stream joins, and custom stateful operations (`flatMapGroupsWithState` / `applyInPandasWithState`).

Operations Not Supported in Streaming

Certain operations that are straightforward in batch processing cannot be applied to unbounded streaming DataFrames because they are semantically undefined on infinite data or require global knowledge:

- **Exact count / count distinct**: These require seeing all data before producing a result. In streaming mode, 'all data' never arrives. Use window-based counts or approximate distinct counts instead.
- **Global sort / orderBy**: A globally sorted stream would require buffering all events before producing any output. Use window-based ranking (`row_number` over a window) for ordered results.
- **Limit**: Limiting to the first N rows of a stream is undefined — which N rows? Use stateful aggregations with output mode 'complete' for small result tables.

- Multiple stateful aggregations chained in one query: Stacking two groupBy-window operations on top of each other in a single streaming query is not supported. Work around this by using foreachBatch to apply the second aggregation in a separate batch operation.

2.20 Input Sources and Output Sinks

Input Sources

An input source is the origin of data for a streaming query. Structured Streaming provides a pluggable source API, and several built-in sources are available:

Apache Kafka Source

Apache Kafka is the most widely used production source for Structured Streaming. Kafka is a distributed, partitioned, replicated log that provides ordered, durable, replayable streams. Each topic in Kafka is divided into partitions, and each partition maintains an ordered, numbered sequence of records (messages). The Structured Streaming Kafka source reads messages from one or more topics, maintaining the read position (offset) in each partition in the checkpoint's offset log. On restart, reading resumes from the checkpointed offsets, ensuring no messages are missed. Kafka's configurable retention period allows data to be replayed from an earlier point if needed.

File Source

The file source watches a directory (on HDFS, S3, or a local filesystem) for new files. When new files are detected, they are processed as a new micro-batch. The file source maintains a list of processed files in the checkpoint to avoid reprocessing. It supports Parquet, CSV, JSON, ORC, and other formats. A schema must always be explicitly provided (schema inference is not supported in streaming file source because it would require scanning files repeatedly). The maxFilesPerTrigger option limits how many new files are processed per micro-batch, preventing overload when many files arrive simultaneously.

Rate Source

The rate source generates synthetic data at a configured rate (rows per second) and is intended for benchmarking and load testing streaming pipelines. It produces two columns: timestamp (event time) and value (a monotonically increasing long integer). It has no dependency on any external system and is always available.

Socket Source

The socket source reads newline-delimited text from a TCP socket. It is intended exclusively for development, debugging, and classroom demonstrations. It has no fault tolerance (data sent before the query starts is lost, and data received during a failure is not replayed) and should never be used in production.

Output Sinks

An output sink is where the results of a streaming query are written. The choice of sink affects the achievable delivery semantics:

File Sink

Writes output to files (Parquet, CSV, JSON, ORC) on a distributed filesystem (HDFS, S3). The file sink achieves exactly-once semantics by writing output files to a temporary location and atomically renaming them to the final location only after the entire micro-batch has been written successfully. If the query fails mid-batch, the incomplete temporary files are discarded and the batch is retried from scratch. The file sink only supports append output mode. It can write partitioned output (by date, hour, or any column) which helps organise the output for efficient downstream reading.

Apache Kafka Sink

Writes output records to a Kafka topic. The output DataFrame must contain a value column (string or binary) and optionally a key column and a topic column (to route different records to different topics). The Kafka sink provides at-least-once semantics by default — on failure and retry, records may be written to Kafka more than once. Exactly-once semantics require Kafka's transactional producer API (supported from Spark 3.0+ with appropriate Kafka version), which groups all writes in a micro-batch into an atomic transaction.

Console Sink

Prints the output of each micro-batch to the driver's standard output (console). Supports both append and update output modes. Designed for development, debugging, and classroom demonstrations only. It has no fault tolerance (output is lost if the driver restarts) and cannot be used as a production sink.

Delta Lake Sink

Writes output to a Delta Lake table, which is an open-source storage format that adds ACID transaction semantics on top of Parquet files. Delta Lake provides exactly-once semantics for streaming writes, supports all three output modes, and enables powerful use cases like streaming MERGE/upsert operations through the foreachBatch pattern. Delta Lake is particularly well-suited for use cases where streaming writes must coexist with batch reads and writes on the same table, or where updates and deletes are required.

ForeachBatch Sink

The foreachBatch sink allows the user to run arbitrary batch processing logic on each micro-batch of streaming output. Instead of writing to a fixed destination, the user provides a function that receives the output DataFrame for each micro-batch as a regular (non-streaming) DataFrame along with a batch ID. Inside this function, any batch DataFrame operation can be applied — writing to a JDBC database, performing a Delta MERGE, calling an external API, applying a second level of aggregation, or writing to multiple destinations simultaneously. ForeachBatch is the most flexible output sink and serves as the bridge between the streaming engine and any destination that does not have a native Structured Streaming sink.

Foreach Sink

The foreach sink provides a row-level callback mechanism: the user defines an object (implementing the ForeachWriter interface) with open, process, and close methods. Spark calls these methods for each partition of each micro-batch. This is lower-level than foreachBatch and is useful when writing to a system that requires connection management (open a connection per partition, write rows one by one, close the connection). However, foreachBatch is generally preferred for its greater flexibility.